# PYTHON ADVANCED & CORE CONCEPTS

## A Mechanism-Driven Handbook on Python Language Internals, Functional Tools, Scopes & Engineering Standards

*A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook*

| METADATA FIELD | SPECIFICATION |
|---|---|
| **Subject:** | Python Internals & Multimedia Standards |
| **Target Audience:** | SDE / Technical Interviews (Top Tech, FAANG, Product Firms) |
| **Methodology:** | Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs |
| **Source Files Consolidated:** | Python Advanced Concepts/*.py & Interview Concepts/GeneralKnowledge.py |
| **Subject Coverage:** | Decorators, Lambda & Zip, Global vs Nonlocal, Python vs Java Collections, Mutability, Video & Image Standards |
| **Key Architectural Focus:** | Function Closures, Hashability Invariants, LEGB Scope Resolution & Compression Formats |
| **Target Evaluation:** | Technical Interview Placement Preparation - Core Engineering Track |

| CH | CHAPTER TITLE & CORE MECHANISM | CRITICAL INTERVIEW CONCEPTS & CODE RECIPES |
|---|---|---|
| **1** | Python Decorators & Higher-Order Functions | Function closures, `@decorator` syntactic sugar, wrapper functions, execution flow |
| **2** | Iterables & Functional Tools: `zip()`, `map()`, `lambda` | `zip()` shortest-termination invariant, anonymous lambda expressions, inline sorting |
| **3** | Variable Scopes in Python: LEGB, `global` vs `nonlocal` | Local, Enclosing, Global, Built-in; Preventing recursion state corruption |
| **4** | Python Collections Internals & Hashability Invariants | list vs tuple vs set vs dict; Mutability, hash lookup O(1) mathematical rules |
| **5** | Cross-Language Collections: Python vs Java | Python list/set/dict mapped to Java ArrayList, HashSet, HashMap, TreeSet, TreeMap |
| **6** | Bitwise Operations & Number Base Conversions | Power-of-two bit tricks (`n & (n - 1)`), `bin()`, `hex()`, `int(x, base)` conversions |
| **7** | Digital Multimedia Standards: Video Resolutions | 240p to 8K UHD, Progressive scan 'p', Aspect Ratios (16:9), DCI 2K vs QHD 1440p |
| **8** | Digital Image Standards: JPEG vs PNG Compression | Lossy DCT compression (photographic) vs Lossless DEFLATE (sharp edges, alpha channel) |

# CHAPTER 1 - PYTHON DECORATORS & HIGHER-ORDER FUNCTIONS

A **Decorator** is a higher-order function that takes another function as an argument, extends or modifies its behavior, and returns a replacement callable object without mutating the original function's source code.

## 1.1 The Syntactic Sugar Mechanism

Writing @my_decorator directly above def func(): pass is purely syntactic sugar for: func = my_decorator(func). The decorator executes at function definition time (import time), returning a closure wrapper.

**Code: Production Decorator with @wraps**

```
import time
from functools import wraps

def execution_timer(func):
    @wraps(func)  # Preserves func's __name__, __doc__, and metadata
    def wrapper(*args, **kwargs):
        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        duration = time.perf_counter() - start_time
        print(f"[BENCHMARK] {func.__name__} executed in {duration:.6f}s")
        return result
    return wrapper

@execution_timer
def compute_squares(n):
    return [i * i for i in range(n)]
```

# CHAPTER 2 - ITERABLES & FUNCTIONAL TOOLS: ZIP(), MAP(), LAMBDA

## 2.1 The zip() Function Mechanics & Termination Invariant

zip(*iterables) takes elements from multiple iterables index-by-index and yields tuples. *Critical Invariant:* **zip() stops unconditionally when the shortest iterable ends!** If you need to iterate to the longest iterable, use itertools.zip_longest(..., fillvalue=None).

## 2.2 Lambda Anonymous Functions & Custom Sorting

A lambda expression is a single-expression anonymous function with implicit return: lambda args: expression. Unlike named functions, it cannot contain statements (e.g., while, return, assignments).

**Code: Lambda Sorting & Tie-Breakers**

```
# Custom Multi-Attribute Sorting with Lambdas
students = [("Alice", 88, 20), ("Bob", 95, 19), ("Charlie", 88, 22)]

# Sort by grade DESCENDING, then by age ASCENDING (Tie-Breaker):
students.sort(key=lambda x: (-x[1], x[2]))

# Sorting Dictionary by Values:
scores = {"Alice": 88, "Bob": 95, "Charlie": 72}
sorted_by_val = sorted(scores.items(), key=lambda x: x[1], reverse=True)
# sorted() returns a new list of tuples: [('Bob', 95), ('Alice', 88), ('Charlie', 72)]
```

# CHAPTER 3 - VARIABLE SCOPES: LEGB, GLOBAL VS NONLOCAL

## 3.1 The LEGB Rule of Scope Resolution

When referencing a variable name, Python searches four concentric scope namespaces in strict sequence:

1. **L - Local:** Names assigned within the current function body.
2. **E - Enclosing:** Names in the local scope of any enclosing/outer functions (closures).
3. **G - Global:** Names assigned at the top level of the current module file.
4. **B - Built-in:** Names pre-assigned in the Python runtime builtins (len, range, int).

## 3.2 global vs nonlocal Invariant

| Keyword | Target Scope Level | Use Case in DSA / Algorithms | Consequence if Omitted on Assignment |
|---|---|---|---|
| None (Default) | Local | Standard temporary variables. | If you assign to a variable, Python marks it local, causing UnboundLocalError if read first! |
| nonlocal | **Enclosing** (Outer function) | Modifying accumulator counters or best-path state in nested DFS/backtracking. | Inner function cannot mutate enclosing state. |
| global | **Module Global** | Modifying module-level singleton state. | Creates a shadow local variable instead of mutating the global. |

# CHAPTER 4 - PYTHON COLLECTIONS & HASHABILITY INVARIANTS

| Collection Type | Ordered? | Mutable? | Duplicates? | Hashable (Valid Dict Key / Set Item)? |
|---|---|---|---|---|
| list | YES (Index-ordered) | YES | YES | **NO (Unhashable type: 'list')** |
| tuple | YES (Index-ordered) | **NO (Immutable)** | YES | **YES** (if all elements inside are hashable!) |
| set | NO (Hash-bucket order) | YES | **NO (Strictly unique)** | **NO** (frozenset is the immutable hashable variant) |
| dict | YES (Insertion order 3.7+) | YES | Keys unique; Values duplicate | Keys MUST be hashable; Values can be any object |

**WHY ARE SETS AND DICTIONARIES O(1)?**

Python sets and dictionaries are implemented as open-addressing hash tables. When an element is looked up, Python calculates hash(x) and masks it to directly index a memory bucket array. *The Invariant:* An object is only hashable if its hash value never changes during its lifetime (requires immutability). Lists cannot be hashed because modifying a list would change its hash, leaving it stranded in the wrong hash bucket!

# CHAPTER 5 - CROSS-LANGUAGE COLLECTIONS: PYTHON VS JAVA

| Feature / Property | Python Data Structure | Java Collection Equivalent | Time Complexity |
|---|---|---|---|
| Dynamic Array | `list` | `ArrayList` | O(1) append/index, O(N) insert/delete at front |
| Doubly Linked Deque | `collections.deque` | `ArrayDeque` / `LinkedList` | O(1) push/pop at both ends |
| Unique Hash Set | `set` | `HashSet` | O(1) lookup, insertion, deletion |
| Sorted Balanced Set | `sortedcontainers.SortedSet` | `TreeSet` (Red-Black Tree) | O(log N) lookup, range queries |
| Key-Value Hash Map | `dict` | `HashMap` | O(1) get, put, containsKey |
| Sorted Key-Value Map | SortedDict / OrderedDict | `TreeMap` (Red-Black Tree) | O(log N) sorted key lookups |
| Min Priority Queue | `heapq` (module over list) | `PriorityQueue` (Min-Heap) | O(log N) insert/extract-min, O(1) peek |

# CHAPTER 6 - BITWISE OPERATIONS & NUMBER BASE CONVERSIONS

## 6.1 Essential Number Base Conversion Recipes (from day23)

**Code: Base Conversions & Power of 2**

```
# 1. Parsing Strings to Integers with Explicit Base
decimal_val = int('1011', 2)    # Binary base 2 -> 11
hex_val     = int('1A', 16)     # Hexadecimal base 16 -> 26
octal_val   = int('77', 8)      # Octal base 8 -> 63

# 2. Converting Integers to Formatted Binary/Hex Strings
bin_str = bin(9)                # '0b1001' (has '0b' prefix)
bin_clean = bin(9)[2:]          # '1001' (sliced to remove prefix!)
hex_str = hex(255)[2:]          # 'ff'

# 3. Mathematical vs Bitwise Power of Two Verification
# Naive approach: (math.log(n, 2)) % 1 == 0 (prone to floating point imprecision!)
# Optimal O(1) Bitwise approach:
def is_power_of_two(n: int) -> bool:
    return n > 0 and (n & (n - 1)) == 0
```

# CHAPTER 7 - DIGITAL MULTIMEDIA STANDARDS: VIDEO RESOLUTIONS

| Quality Name | Pixel Resolution | Aspect Ratio | Standard Category | Vertical Scan Meaning |
|---|---|---|---|---|
| 240p | 426 x 240 | 16:9 | Low Definition (Mobile save) | 240 vertical progressive scan lines |
| 360p | 640 x 360 | 16:9 | Low Definition | 360 vertical progressive scan lines |
| 480p | 854 x 480 | 16:9 | Standard Definition (SD / DVD) | 480 vertical progressive scan lines |
| 720p | **1280 x 720** | 16:9 | **High Definition (HD)** | 720 vertical scan lines |
| 1080p | **1920 x 1080** | 16:9 | **Full HD (FHD)** | 1080 vertical scan lines (1080p60 standard) |
| 1440p (QHD / 2K) | **2560 x 1440** | 16:9 | **Quad HD (4x 720p pixels!)** | 2560 horizontal x 1440 vertical |
| 2160p (4K UHD) | **3840 x 2160** | 16:9 | **Ultra HD (4K UHD)** | 3840 horizontal x 2160 vertical (4x 1080p!) |
| 4320p (8K UHD) | **7680 x 4320** | 16:9 | **8K Ultra HD** | 7680 horizontal x 4320 vertical (33.2 Megapixels) |

### 7.2 Critical Distinction: DCI 2K / 4K vs Consumer 1440p / UHD

• **DCI 2K (Cinema standard):** 2048 x 1080 vs **QHD 1440p (Consumer displays):** 2560 x 1440.
• **DCI 4K (Cinema standard):** 4096 x 2160 vs **4K UHD (Consumer TV / YouTube):** 3840 x 2160.
• The p stands for **Progressive Scan** (every line drawn sequentially per refresh) as opposed to i (Interlaced Scan: alternating odd/even lines).

# CHAPTER 8 - DIGITAL IMAGE STANDARDS: JPEG VS PNG COMPRESSION

| Dimension | JPEG (Joint Photographic Experts Group) | PNG (Portable Network Graphics) |
|---|---|---|
| Compression Type | **Lossy Compression:** Discards high-frequency color variations invisible to the human eye. | **Lossless Compression:** Preserves 100% of original pixel data and color values. |
| Compression Algorithm | Discrete Cosine Transform (DCT) + Quantization + Huffman Coding. | DEFLATE (LZ77 algorithm + Huffman coding, identical to gzip). |
| Color Depth | 24-bit Truecolor (16.7 Million colors). | 24-bit or 32-bit (supports 8-bit Alpha Channel). |
| Alpha Transparency | **STRICTLY NO** (Cannot render transparent backgrounds!). | **YES** (Supports full partial and gradient alpha transparency). |
| Artifacts on Re-save | Suffers from **Generation Loss** (re-saving degrades quality with ringing artifacts). | Zero quality degradation on repeated saves. |
| Ideal Use Case | Real-world photographs, nature landscapes, complex multi-color imagery. | Logos, UI icons, screenshots, technical diagrams, sharp text graphics. |

### PYTHON & ENGINEERING STANDARDS CHEAT-SHEET

- **Decorators:** Closures wrapping callables; always use `@wraps(func)` to preserve function metadata.
- **Scope Resolution:** LEGB (Local -> Enclosing -> Global -> Built-in); use `nonlocal` for nested DFS state.
- **Mutability Rule:** Only immutable types (int, str, tuple) can be hash table keys (dict/set).
- **Video Standards:** 1080p = 1920x1080; 1440p = 2560x1440; 4K UHD = 3840x2160.
- **Image Formats:** JPEG is lossy DCT for photos; PNG is lossless DEFLATE for graphics and transparency.